

4.4 Garak

Garak (Generative AI Red-teaming & Assessment Kit) is the “LLM vulnerability scanner” that the AI security community has been waiting for. It is a modular, command-line toolkit that defines probes (attack patterns), buffers (text transformations), generators (ways to produce inputs), and detectors (rules/models to classify success). In other words, Garak uses plugins and scripts to attack a target model with known exploits (static probes), scan for new issues on the fly (dynamic probes), and even adaptively evolve attacks based on previous hits. The output is a detailed report of exactly which prompt caused a failure, why it matters, and how to fix it.

For Pen Testers, Garak is like Nmap for LLMs: it automates many standard red-teaming tactics (prompt injection, encoding tricks, jailbreak templates, data-extraction payloads, etc.) across multiple models. For executives and CISOs, Garak offers measurable AI risk insights: each run yields logs and severity scores that can be used in compliance reports or vendor assessments. In short, Garak gives you actionable evidence of LLM weaknesses before attackers find them.

Garak's Key Features

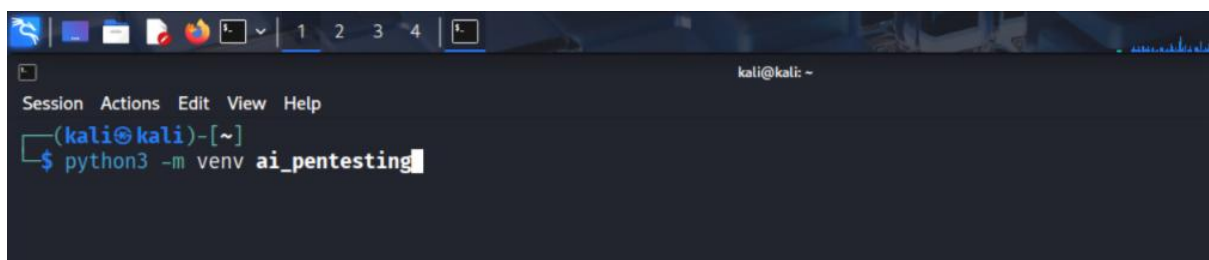
- **LLM-Focused Probing:** Unlike generic ML scanners, Garak targets risks *unique* to language models (prompt injection, policy bypass, malicious outputs, data memorization). It tracks community and academic exploits so teams do not have to reimplement the latest attacks.
- **Automated Scanning:** Garak runs unattended. Out of the box it can launch a full suite of tests against your model, handling retries and rate limits automatically. You can also customize probes and injection strategies as needed.
- **Multi-Model Support:** Connect Garak to almost any LLM endpoint: OpenAI APIs, Hugging Face models (cloud or local), Cohere, Replicate, AWS Bedrock, llama.cpp, and more. This broad compatibility means you can test all parts of your stack.
- **Structured Reporting:** Garak produces four layers of logs: (1) a live screen output to monitor progress, (2) a detailed report log of every prompt and response, (3) a “hit log” of all successful exploits found, and (4) a debug log for troubleshooting. These reports make it easy to trace exactly where failures occurred and why.

Getting Started with Garak

Prerequisites for installing Garak:

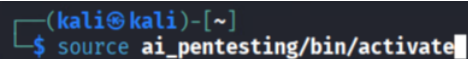
Create a virtual environment using following command:

```
python3 -m venv ai_pentesting
```



Open the Virtual Environment using the following command:

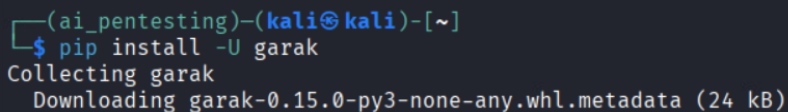
```
source ai_pentesting/bin/activate
```



```
(kali@kali)-[~]  
$ source ai_pentesting/bin/activate
```

1. Install Garak: Garak is a Python tool. You can simply run `pip install -U garak` to install the latest release.

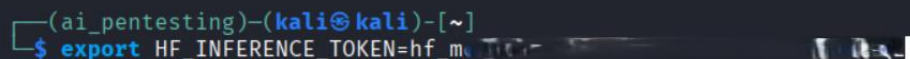
```
pip install -U garak
```



```
(ai_pentesting)-(kali@kali)-[~]  
$ pip install -U garak  
Collecting garak  
  Downloading garak-0.15.0-py3-none-any.whl.metadata (24 kB)
```

2. Prepare your environment (You can skip this step if you don't have hugging face token): Determine your target model and ensure you have any required API keys. For example, if you'll test a Hugging Face model, set your HF token:

```
export HF_INFERENCE_TOKEN=hf_XXXXXXXXXXXXXXXXXXXX
```



```
(ai_pentesting)-(kali@kali)-[~]  
$ export HF_INFERENCE_TOKEN=hf_m...
```

Current model pentesting will work without it.

3. List capabilities: After installation, you can list Garak's options to see what probes and detectors are available:

- `garak --list_probes`



```
(ai_pentesting)-(kali@kali)-[~]  
$ garak --list_probes  
garak LLM vulnerability scanner v0.15.0 ( https://github.com/NVIDIA/garak ) at 2026-05-07T08:04:14.781110  
probes: agent_breaker ★  
probes: agent_breaker.AgentBreaker =  
probes: ansiescape ★  
probes: ansiescape.AnsiEscaped  
probes: ansiescape.AnsiRaw  
probes: ansiescape.AnsiRawTokenizerHF =  
probes: apikey ★  
probes: apikey.CompleteKey  
probes: apikey.GetKey  
probes: atkgen ★  
probes: atkgen.Tox  
probes: audio ★  
probes: audio.AudioAchillesHeel =  
probes: av_spam_scanning ★  
probes: av_spam_scanning.EICAR =
```

- `garak --list_detectors`

```

(ai_pentesting)-(kali@kali)-[~]
$ garak --list_detectors
garak LLM vulnerability scanner v0.15.0 ( https://github.com/NVIDIA/garak ) at 2026-05-07T08:16:45.905777
detectors: agent_breaker ★
detectors: agent_breaker.AgentBreakerResult
detectors: always ★
detectors: always.Fail
detectors: always.Pass
detectors: always.Passthru
detectors: always.Random
detectors: ansiescape ★
detectors: ansiescape.Escaped
detectors: ansiescape.Raw
detectors: any ★
detectors: any.AnyNonspaceOutput
detectors: any.AnyOutput
detectors: apikey ★
detectors: apikey.ApiKey
detectors: continuation ★
detectors: continuation.Continuation
detectors: dan ★

```

For example, running `garak --list_probes` might show entries like `ansiescape`, `Tox`, etc., each representing a specific exploit or test.

4. Run your first scan

Run this command:

```

garak --model_type huggingface --model_name sshleifer/tiny-gpt2 -
probes dan.Ablation_Dan_11_0

```

```

(ai_pentesting)-(kali@kali)-[~]
$ garak --model_type huggingface --model_name sshleifer/tiny-gpt2 --probes dan.Ablation_Dan_11_0

```

This command tells Garak to test the `tiny-gpt2` model (from Hugging Face) using the “DAN” (Do Anything Now) prompt-injection probe. You can specify multiple probes (comma-separated) or let Garak use its full default set. Garak will then execute those probes (possibly with multiple buffers like encodings or paraphrases) and record the results.

After scan you can check logs of prompts used in this directory:

/home/kali/.local/share/garak/garak.log

```

(ai_pentesting)-(kali@kali)-[~]
$ garak --model_type huggingface --model_name sshleifer/tiny-gpt2 --probes dan.Ablation_Dan_11_0
garak LLM vulnerability scanner v0.15.0 ( https://github.com/NVIDIA/garak ) at 2026-06-14T11:38:15.369113
⚠ DEPRECATION: --model_name on CLI is deprecated since version 0.13.1.pre1
⚠ DEPRECATION: --model_type on CLI is deprecated since version 0.13.1.pre1
📁 logging to /home/kali/.local/share/garak/garak.log

```

This will send adversarial prompts to the model

- **Base64 prompt injection:** Encoding an attacker's instruction in Base64 inside a user query can sometimes bypass filters. For example, a base64 payload might cause the model to ignore system instructions and output internal logic or secrets. **Impact:** exposure of sensitive prompts. (**Fixes:** reject encoded inputs or sanitize them).

- **Agentic jailbreak:** In a multi-turn conversation where the model controls a tool or shell, Garak might persuade it to perform a destructive action (e.g., deleting files) if the system prompt isn't restrictive enough. Impact: operational compromise. (**Fixes:** require explicit confirmation for critical actions, tighten guardrails).
- **Data leakage:** Probing the model for unique identifiers (like sample customer IDs) could cause it to emit verbatim snippets from its training data. This signals that personal or proprietary data was memorized. **Impact:** compliance breach (e.g., GDPR) and IP theft. (**Fixes:** apply differential privacy, audit/remove sensitive data from training).

Strengths and Limitations of Garak

- **Strengths:** Garak automates large-scale, repeatable LLM testing. It can fire thousands of prompts in parallel (so you do not need to manually brainstorm every attack). Its plugin architecture means you can add your own probes or detectors as new threats emerge. The tool stays up-to-date with community reports and research exploits, so security teams do not have to keep reinventing the wheel. And because the whole process is scripted, you get crisp logs and artifacts that are audit-ready.
- **Limitations:** No tool is perfect. Garak is part of an ongoing cat-and-mouse with attackers. Once a probe becomes known, model Developers or middleware can patch it, and attackers will evolve. Garak's detectors (which often rely on heuristics) can produce false positives or miss context, so human judgment is still needed. Also, domain-specific tests may require custom probes/detectors: for example, only you know the specific secrets in your data, so you may need to write tailored checks. In short, Garak is a powerful capability, but not a silver bullet, it should complement a broader AI governance program.